Step 1 is a bit involved, but step 2 is easy. In any case, this two-step approach results in a simpler algorithm than trying to parse the arithmetic expression directly. For most humans, of course, it's easier to parse the ordinary arithmetic expression. We return to the difference between the human and computer approaches in a moment.

Before we delve into the details of steps 1 and 2, we introduce the new notation.

Postfix Notation

Everyday arithmetic expressions are written with an **operator** (+, −, ×, or /) placed between two **operands** (numbers, or symbols that stand for numbers). This is called **infix** notation because the operator is written *inside* the operands. Typical arithmetic expressions are things like 2+2 and 4/7, or, using letters to stand for numbers, A+B and A/B.

In **postfix** notation (which is also called Reverse Polish Notation, or RPN, because it was invented by a Polish mathematician), the operator *follows* the two operands. Thus, A+B becomes AB+, and A/B becomes AB/. More complex infix expressions can likewise be translated into postfix notation, as shown in Table 4-2. One thing that might stand out in these translations is that there are no parentheses in the postfix expressions. We explain how the postfix expressions are generated in a moment.

Table 4-2 *Infix and Postfix Expressions*

Infix	Postfix
A+B−C	AB+C−
A×B/C	AB×C/
A+B×C	ABC×+
A×B+C	AB×C+
A×(B+C)	ABC+×
A×B+C×D	AB×CD×+
(A+B)×(C−D)	AB+CD−×
((A+B)×C)−D	AB+C×D−
A+B×(C−D/(E+F))	ABCDEF+/−×+

Besides infix and postfix, there's also a **prefix** notation, in which the operator is written before the operands: +AB instead of AB+. This notation is functionally like postfix but seldom used. The lambda calculus (λ calculus) and the Lisp programming language use prefix notation and are very important historically. For functional programming languages, prefix notation is very convenient. The lambda calculus is also the origin of the `lambda` keyword used for anonymous functions in Python.

Translating Infix to Postfix

The next several pages are devoted to explaining how to translate an expression from infix notation into postfix. This algorithm is fairly involved, so don't worry if every detail isn't clear at first. If you get bogged down, you may want to skip ahead to the sections "The Infix Calculator Tool" and "Evaluating Postfix Expressions"

To understand how to create a postfix expression, you might find it helpful to see how a postfix expression is evaluated; for example, how the value 14 is computed from the expression 234+×, which is the postfix equivalent of 2×(3+4). Notice that in this discussion, for ease of writing, we restrict ourselves to expressions with single-digit numbers (and sometimes variable

names) for inputs. Later, we look at code to handle expressions with multidigit numbers and multicharacter variable names.

How Humans Evaluate Infix

How do you translate infix to postfix? Let's examine a slightly easier question first: How does a human evaluate a normal infix expression? Although, as we stated earlier, such evaluation is difficult for a computer, we humans do it fairly easily because of countless hours looking at similar expressions in math class. It's not hard to find the answer to 3+4+5, or 3×(4+5). By analyzing how you evaluate this expression, you can achieve some insight into the translation of such expressions into postfix.

Roughly speaking, when you "solve" an arithmetic expression, you follow rules something like this:

1. You read from left to right. (At least, we assume this is true. Sometimes people skip ahead, but for purposes of this discussion, you should assume you must read methodically, starting at the left.)
2. When you've read enough to evaluate two operands and an operator, you do the calculation and substitute the answer for these two operands and operator. (You may also need to solve other pending operations on the left, as we see later.)
3. You continue this process—going from left to right and evaluating when possible—until the end of the expression.

Table 4-3, Table 4-4, and Table 4-5 show three examples of how simple infix expressions are evaluated. Later, Tables 4-6, 4-7, and 4-8, show how closely these evaluations mirror the process of translating infix to postfix.

To evaluate 3+4−5, you would carry out the steps shown in Table 4-3.

Table 4-3 *Evaluating 3+4−5*

Item Read	Expression Parsed So Far	Comments
3	3	
+	3+	
4	3+4	
−	7	When you see the −, you can evaluate 3+4.
	7−	
5	7−5	
End	2	When you reach the end of the expression, you can evaluate 7−5.

You can't evaluate the 3+4 until you see whether an operator follows the 4 and what operator it is. If it's × or /, you need to wait before applying the + sign until you've evaluated the × or /. In this example, however, the operator following the 4 is a −, which has the same **precedence** (priority among operators) as a +, so when you see the −, you know you can evaluate 3+4, which is 7. The 7 then replaces the 3+4. You can evaluate the 7−5 when you arrive at the end of the expression, knowing that there are no more operators.

Because of precedence relationships, evaluating 3+4×5 is a bit more complicated, as shown in Table 4-4.

Table 4-4 *Evaluating 3+4×5*

Item Read	Expression Parsed So Far	Comments
3	3	
+	3+	
4	3+4	
×	3+4×	You can't evaluate 3+4 because × is higher precedence than +.
5	3+4×5	Because you don't know if there is some higher-precedence expression that follows, you simply put the 5 at the end.
End	3+20	When you see there are no more expressions, you can evaluate 4×5.
	23	Because there are still operators, you now evaluate 3+20.

Here, you can't add the 3 until you know the result of 4×5. Why not? Because multiplication has a higher precedence than addition. In fact, both × and / have a higher precedence than + and −, so all multiplications and divisions must be carried out before any additions or subtractions (unless parentheses dictate otherwise; see the next example).

Often you can evaluate as you go from left to right, as in the preceding example. You need to be sure, when you come to an operand-operator-operand combination such as A+B, however, that the operator on the right side of the B isn't one with a higher precedence than the +. If it does have a higher precedence, as in this example, you can't do the addition yet. After you've read the 5 and found nothing after it, however, you know the multiplication can be carried out. Note that because multiplication has the highest priority among the four operators in this sample language; it doesn't matter whether a × or / follows the 5, and the multiplication could proceed without looking at the next input. You can't do the addition, however, until you've found out what's beyond the 5. When you find there's nothing more to read, you can go ahead and perform any remaining operations, knowing the highest precedence operators are on the right.

Parentheses are used to override the normal precedence of operators. Table 4-5 shows how you would evaluate 3×(4+5). Without the parentheses, you would do the multiplication first; with them, you do the addition first.

Table 4-5 *Evaluating 3×(4+5)*

Item Read	Expression Parsed So Far	Comments
3	3	
×	3×	
(	3×(	
4	3×(4	You can't evaluate 3×4 because of the parenthesis.
+	3×(4+	
5	3×(4+5	You can't evaluate 4+5 yet.
)	3×(4+5)	When you see the), you can evaluate 4+5.
	3×9	After replacing the parenthesized expression, you need to know if there's more to come with a higher precedence.
End	27	There isn't, so now you evaluate 3×9.

Here, you can’t evaluate anything until you’ve reached the closing parenthesis. Multiplication has a higher or equal precedence compared to the other operators, so ordinarily you could carry out 3×4 as soon as you see the 4. Parentheses, however, have an even higher precedence than × and /. Accordingly, you must evaluate anything in parentheses before using the result as an operand in any other calculation. The closing parenthesis tells you that you can go ahead and do the addition. You find that 4+5 is 9 and substitute it. Finding that there are no more higher-precedence operators afterward, you can evaluate 3×9 to obtain 27.

As you’ve seen in evaluating an infix arithmetic expression, you go both forward and backward through the expression. You go forward (left to right)

reading operands and operators. When you have enough information to apply an operator, you go backward, recalling two operands and an operator and carrying out the arithmetic.

Sometimes you must defer applying operators if they're followed by higher-precedence operators or by parentheses. When this happens, you must apply the later, higher-precedence, operator first; then go backward (to the left) and apply earlier operators.

You could write an algorithm to carry out this kind of evaluation directly. As we noted, however, it's easier to translate into postfix notation first.

How Humans Translate Infix to Postfix

To translate infix to postfix notation, you follow a similar set of rules to those for evaluating infix. There are, however, a few small changes. You don't do any arithmetic. The idea is not to evaluate the infix expression, but to rearrange the operators and operands into a different format: postfix notation. The resulting postfix expression will be evaluated later.

As before, you read the infix from left to right, looking at each character in turn. As you go along, you copy these operands and operators to the postfix output string. The trick is knowing when to copy what.

If the character in the infix string is an operand, you copy it immediately to the postfix string. That is, if you see an A in the infix, you write an A to the postfix. There's never any delay: you copy the operands as you get to them, no matter how long you must wait to copy their associated operators.

Knowing when to copy an operator is more complicated, but it's the same as the rule for evaluating infix expressions. Whenever you could have used the operator to evaluate part of the infix expression (if you were evaluating instead of translating to postfix), you instead copy it to the postfix string.

Table 4-6 shows how A+B−C is translated into postfix notation.

Table 4-6 *Translating A+B−C into Postfix*

Character Read from Infix Expression	Infix Expression Parsed So Far	Postfix Expression Written So Far	Comments
A	A	A	
+	A+	A	
B	A+B	AB	
−	A+B-	AB+	When you see the −, you can copy the + to the postfix string.
C	A+B−C	AB+C	
End	A+B−C	AB+C−	When you reach the end of the expression, you can copy the −.

Notice the similarity of this table to Table 4-3, which showed the evaluation of the infix expression 3+4−5. At each point where you would have done an evaluation in the earlier table, you instead simply write an operator to the postfix output.

Table 4-7 shows the translation of A+B×C to postfix. This translation parallels that of Table 4-4, which covered the evaluation of 3+4×5.

Table 4-7 *Translating A+B×C into Postfix*

Character Read from Infix Expression	Infix Expression Parsed So Far	Postfix Expression Written So Far	Comments
A	A	A	
+	A+	A	
B	A+B	AB	
C	A+B×C	ABC	When you see the C, you can copy the ×.
	A+B×C	ABC×	
End	A+B×C	ABC×+	When you see the end of the expression, you can copy the +.

As the final example, Table 4-8 shows how A×(B+C) is translated to postfix. This process is similar to evaluating 3×(4+5) in Table 4-5. You can't write any postfix operators until you see the closing parenthesis in the input.

Table 4-8 *Translating A×(B+C) into Postfix*

Character Read from Infix Expression	Infix Expression Parsed so Far	Postfix Expression Written So Far	Comments
A	A	A	
×	A×	A	
(	A×(	A	
B	A×(B	AB	You can't copy × because of the parenthesis.
+	A×(B+	AB	
C	A×(B+C	ABC	You can't copy the + yet.
)	A×(B+C)	ABC+	When you see the), you can copy the +.
	A×(B+C)	ABC+×	After you've copied the +, you can copy the ×.
End	A×(B+C)	ABC+×	Nothing left to copy.

As in the numerical evaluation process, you go both forward and backward through the infix expression to complete the translation to postfix. You can't write an operator to the output (postfix) string if it's followed by a higher-precedence operator or a left parenthesis. If it is, the higher-precedence operator or the operator in parentheses must be written to the postfix before the lower-priority operator.

Saving Operators on a Stack

You'll notice in both Table 4-7 and Table 4-8 that the order of the operators is reversed going from infix to postfix. Because the first operator can't be copied to the output until the second one has been copied, the operators were output to the postfix string in the opposite order they were read from the infix string. That suggests a stack might be useful for handling the operators. A longer

example helps illustrate how that could work. Table 4-9 shows the translation to postfix of the infix expression A+B×(C−D).

Table 4-9 *Translating A+B×(C−D) into Postfix*

Character Read from Infix Expression	Infix Expression Parsed So Far	Postfix Expression Written So Far	Stack Contents
A	A	A	
+	A+	A	+
B	A+B	AB	+
×	A+B×	AB	+×
(	A+B×(	AB	+×(
C	A+B×(C	ABC	+×(
−	A+B×(C−	ABC	+×(−
D	A+B×(C−D	ABCD	+×(−
)	A+B×(C−D)	ABCD−	+×(
	A+B×(C−D)	ABCD−	+×(
	A+B×(C−D)	ABCD−	+×
	A+B×(C−D)	ABCD−×	+
	A+B×(C−D)	ABCD−×+	

Here you see the order of the operands is +×− in the original infix expression, but the reverse order, −×+, in the final postfix expression. This happens because × has higher precedence than +, and −, because it's in parentheses, has higher precedence than ×. We need to track operators with lower precedence on the stack to be able to process them later. The last column in Table 4-9 shows the stack contents at various stages in the translation process.

Popping items from the stack allows you to, in a sense, go backward (right to left) through the input string. You're not really examining the entire input string, only the operators and parentheses. They were pushed on the stack when reading the input, so now you can recall them in reverse order by popping them off the stack.

The operands (A, B, and so on) appear in the same order in infix and postfix, so you can write each one to the output as soon as you encounter it; they don't need to be stored on a stack or reversed. Changing their order would also cause issues with operators that lack the commutative property like subtraction and division; $A - B \neq B - A$.

Translation Rules

Let's make the rules for infix-to-postfix translation more explicit. You read items from the infix input string and take the actions shown in Table 4-10. These actions are described in **pseudocode**, a blend of programming syntax and English.

In this table, the < and >= symbols are applied to the operator precedence, not numerical values. The *inputOp* operator has just been read from the infix input, while the *top* operator is popped off the stack. We use prec(*inputOp*) and prec(*top*) to mean the operator precedence of *inputOp* and *top*, respectively.

Table 4-10 *Infix to Postfix Translation Rules*

Item Read from Input	Action (Infix)
Operand	Write operand to postfix output string.
Open parenthesis (	Push parenthesis on stack.
Close parenthesis)	While stack is not empty: *top* = pop item from stack. If *top* is (, then break out of loop. Else write *top* to postfix output.
Operator (*inputOp*)	While stack is not empty: *top* = pop item from stack. If *top* is (, then push (back on stack and break. Else if *top* is an operator: If prec(*top*) >= prec(*inputOp*), output *top*. Else push top and break loop. Push *inputOp* on stack.
End of input	While stack is not empty: Pop stack and output item.

Convincing yourself that these rules work may take some effort. Tables 4-11, 4-12, and 4-13 show how the rules apply to the three sample infix expressions, similar to Tables 4-6, 4-7, and 4-8, except that the relevant rules for each step have been added. Try creating similar tables by starting with other simple infix expressions and using the rules to translate some of them to postfix.

Table 4-11 *Translation Rules Applied to A+B−C*

Character Read from Infix	Infix Parsed So Far	Postfix Written So Far	Stack Contents	Rule
A	A	A		Write operand to output.
+	A+	A	+	While stack not empty: (null loop) Push *inputOp* on stack.
B	A+B	AB	+	Write operand to output.
−	A+B−	AB		Stack not empty, so pop item +.
	A+B−	AB+		*inputOp* is −, *top* is +, prec(*top*) >= prec(*inputOp*), so output *top*.
	A+B−	AB+	−	Then push *inputOp*.
C	A+B−C	AB+C	−	Write operand to output.
End	A+B−C	AB+C−		Pop leftover item, output it.

Table 4-12 *Translation Rules Applied to A+B×C*

Character Read From Infix	Infix Parsed So Far	Postfix Written So Far	Stack Contents	Rule
A	A	A		Write operand to postfix.
+	A+	A	+	If stack empty, push *inputOp*.
B	A+B	AB	+	Write operand to output.
×	A+B×	AB		Stack not empty, so pop *top* +.
	A+B×	AB	+	*inputOp* is ×, *top* is +, prec(*top*) < prec(*inputOp*), so push *top*.
	A+B×	AB	+×	Then push *inputOp*.
C	A+B× C	ABC	+×	Write operand to output.
End	A+B× C	ABC×	+	Pop leftover item, output it.
	A+B× C	ABC×+		Pop leftover item, output it.

Table 4-13 *Translation Rules Applied to A×(B+C)*

Character Read From Infix	Infix Parsed So Far	Postfix Written So Far	Stack Contents	Rule
A	A	A		Write operand to postfix.
×	A×	A	×	If stack empty, push *inputOp*.
(	A×(	A	×(	Push (on stack.
B	A×(B	AB	×(	Write operand to postfix.
+	A×(B+	AB	×	Stack not empty, so pop item (.
	A×(B+	AB	×(	*top* is (, so push it and break.
	A×(B+	AB	×(+	Then push *inputOp*.
C	A×(B+C	ABC	×(+	Write operand to postfix.
)	A×(B+C)	ABC+	×(	Pop item, write to output.
	A×(B+C)	ABC+	×	Quit popping if (.
End	A×(B+C)	ABC+×		Pop leftover item, output it.

The Infix Calculator Tool

Before we look at the code, let's put these words into action and watch the process of converting infix expressions to postfix using a visualization tool. Follow the instructions in Appendix A to launch the InfixCalculator tool. The interface is simple: select the text entry box in the Operations area, type a numeric expression using infix, and select the Evaluate button. The animation of parsing the expression begins and looks something like Figure 4-11.

Figure 4-11 *The Infix Calculator tool*

The expression being parsed in the figure, 2 * (3 + 4), is copied in the box at the top of the tool and then reduced as the characters are processed. (The full expression still appears in the text entry box below.) The figure shows that the first operand, 2, the multiplication operator, *, and the open parenthesis have already been pulled out of the expression and placed in the stack or queue. Each one of those strings is called a `token`. Looking at the "3" token corresponds to the fourth step detailed in Table 4-13 when the B operand is examined.

Because the current token "3" is a number, it's not an operator or a delimiter like those shown in the Operator Precedence table at the upper right. When the tool looks in that table, it finds that numbers have no precedence and writes `prec = None` and `delim = False` to indicate its status as an operand. The next (nonblank) character in the expression is a plus (+). It does have a precedence in the table, 4, and that means it's an operator.

As the animation runs, the stack contents build up in the stack on the left. Tokens that can be output as the postfix expression go in the queue to its right. The contents of this queue are used to make the postfix string later in the animation. Watch the stack grow as each operator is taken from the front of the expression. Pause the infix calculator and see whether you can predict what will happen next.

The Infix Calculator shows the translation into posftfix, followed by the evaluation of the postfix. Before we look at how the evaluation works, let's look at the code for the parsing and translation. The visualization tool shows this code during the processing.

Python Code to Convert Infix to Postfix

Listing 4-9 shows the beginning of the `PostfixTranslate.py` program, which uses the rules from Table 4-10 to translate an infix expression to a postfix expression. This is a long program file, so we show it in two parts. The first part provides the processing of the input string, finding the individual tokens in the expression and determining the precedence of operators.

Listing 4-9 *Processing Tokens in* `PostfixTranslate.py`

```
from SimpleStack import Stack
from Queue import Queue

# Define operators and their precedence
# We group single character operators of equal precedence in strings
# Lowest precedence is on the left; highest on the right
# Parentheses are treated as high precedence operators
operators = ["|", "&", "+-", "*/%", "^", "()"]
def precedence(operator): # Get the precedence of an operator
   for p, ops in enumerate(operators):  # Loop through operators
      if operator in ops:               # If found,
         return p + 1                   # return precedence (low = 1)
      # else not an operator, return None

def delimiter(character):  # Determine if character is delimiter
   return precedence(character) == len(operators)

def nextToken(s):             # Parse next token from input string
   token = ""                 # Token is operator or operand
   s = s.strip()              # Remove any leading & trailing space
   if len(s) > 0:             # If not end of input
      if precedence(s[0]):    # Check if first char. is operator
         token = s[0]         # Token is a single char. operator
         s = s[1:]
      else:                  # its an operand, so take characters up
         while len(s) > 0 and not (   # to next operator or space
                precedence(s[0]) or s[0].isspace()):
            token += s[0]
            s = s[1:]
   return token, s  # Return the token, and remaining input string
```

After importing the previous definitions of stacks and queues, the first statement of `PostfixTranslate.py` defines the `operators` that the program

can process. We have expanded the number of operators that can be used. So far, we've only shown examples using the four most common operators, +−×/. This program adds several others available in Python. By putting these operator characters into an array of strings, we have grouped together operators with the same precedence and ordered the groups. Each array cell corresponds to a precedence level. You can look up the precedence of an operator by stepping through the array until you find the character in the string and returning the string's position in the array. Note that we used the asterisk * for the multiplication symbol ×, to be consistent with Python and other programming languages.

The `precedence()` function does the lookup, returning a number for the precedence or `None` if the character is not an operator. The operators are all the single-character operators that Python allows. Parentheses are included in the array of `operators` for convenience. They act like operators when breaking an expression up into tokens. A separate `delimiter()` function tests whether its argument is one of these highest-precedence characters.

A function breaks the input string into **tokens**. A token is a substring that corresponds to exactly one operator, operand, or delimiter in the input expression.

The `nextToken()` function goes through the input string, skipping over any whitespace, and finding the first token at the beginning of the string. It returns that token along with the rest of the string after the token, which is used for the subsequent call to `nextToken()`. The function checks whether a character is an operator or a delimiter by checking whether the `precedence` function returns a number for it (or `None` for operands). All the operators and delimiters in this example are single characters.

For operands, this program allows multicharacter (multidigit) tokens. The `while` loop steps through the input string, `s`, checking whether the first character is an operator or whitespace. If it is not, the first character is added to the output `token` and removed from the input string. After the loop finds either an operator, delimiter, whitespace, or the end of the input string, the token is complete.

The `PostfixTranslate()` function appears in Listing 4-10 and does the main work by allocating a stack to hold the operators (and left parentheses) and a queue to hold the postfix output. As you saw with the InfixCalculator Visualization tool, the operators build up in the stack and are then popped and

moved to the queue. The queue holds the output postfix string with each operand or operator in its own cell. That enables the program to add some whitespace between elements in the output to make it more legible, especially for operands with more than one digit or character.

`PostfixTranslate()` loops over the input expression/formula using a **fencepost loop**—a loop that performs some work on the initial item before entering the main loop body to perform work on all the remaining loop items. In this case, it extracts the first token and then loops until there are no more tokens in the input expression/formula.

After determining whether the current token is an operand, operator, or delimiter, the `PostfixTranslate()` function applies the rules in Table 4-10. The `if delim … elif prec … else` statement separates the processing of the three types. The Python statements are very close to the pseudocode used to describe the rules. This is the heart of the algorithm, so study it closely. The `if delim: …` block handles the parentheses, pushing open parentheses on the stack and then popping off operators between the opening and closing parentheses after they are found.

Listing 4-10 *The `PostfixTranslate()` Function*

```
def PostfixTranslate(formula):  # Translate infix to Postfix
   postfix = Queue(100)      # Store postfix in queue temporarily
   s = Stack(100)            # Parser stack for operators

   # For each token in the formula  (fencepost loop)
   token, formula = nextToken(formula)
   while token:
      prec = precedence(token)  # Is it an operator?
      delim = delimiter(token)  # Is it a delimiter?
      if delim:
         if token == '(':    # Open parenthesis
            s.push(token)    # Push parenthesis on stack
         else:               # Closing parenthesis
            while not s.isEmpty():  # Pop items off stack
               top = s.pop()
               if top == '(':       # Until open paren found
                  break
               else:                # and put rest in output
                  postfix.insert(top)
```

```
        elif prec:           # Input token is an operator
            while not s.isEmpty(): # Check top of stack
                top = s.pop()
                if (top == '(' or    # If open parenthesis, or a lower
                    precedence(top) < prec): # precedence operator
                    s.push(top)      # push it back on stack and
                    break            # stop loop
                else:                # Else top is higher precedence
                    postfix.insert(top) # operator, so output it
            s.push(token)            # Push input token (op) on stack

        else:                        # Input token is an operand
            postfix.insert(token)    # and goes straight to output

        token, formula = nextToken(formula) # Fence post loop

    while not s.isEmpty():           # At end of input, pop stack
        postfix.insert(s.pop())      # operators and move to output

    ans = ""
    while not postfix.isEmpty(): # Move postfix items to string
        if len(ans) > 0:
            ans += " "               # Separate tokens with space
        ans += postfix.remove()
    return ans

if __name__ == '__main__':
    infix_expr = input("Infix expression to translate: ")
    print("The postfix representation of", infix_expr, "is:",
          PostfixTranslate(infix_expr))
```

In the `elif prec: …` block, new operators from the input string are processed. It uses a loop to walk back through the stack looking for open parentheses or lower-precedence operators. The code shows a bit of an optimization from the loop's pseudocode for new operators:

While stack is not empty:

top = pop item from stack

If *top* is (, then push (back on stack and break

Else if *top* is an operator:

If prec(*top*) >= prec(*inputOp*), output *top*

Else push *top* and break loop

The two highlighted phrases in the pseudocode perform the same operation because the open parenthesis is stored in the *top* variable. In the program, the two conditions are checked in the `if (top == '(' or precedence(top) < prec): …` statement. When `top` does not satisfy that condition, it must be an operator whose precedence is greater than or equal to that of the input operator in `token`. That's the only condition where the *top* operator is output by inserting it into the `postfix` queue.

After all the tokens have been processed in the first `while` loop, any remaining operators on the stack are popped and output to the postfix queue, reversing their order. That work happens in the final `while not s.isEmpty()` loop.

At the end, the `while not postfix.isEmpty()` loop creates the `ans` string, by concatenating the operands and operators in the `postfix` output queue separated by spaces. The spaces don't change the value of the expression but do make the string more readable.

After the function body of `PostfixTranslate()` is defined, the last section of the program is an `if` statement that is used to detect whether this file is being used as a program or as a module inside a bigger program. By testing whether the special variable, `__name__`, is set to the string '`__main__`', the Python interpreter can determine whether this file is being loaded as the main file to be executed or as part of an `import` statement inside another file. When `__name__` is '`__main__`' it's the main file to be executed, and this program prompts for an input infix expression to translate. It then prints that expression along with its postfix translation. The output looks like this:

```
$ python3 PostfixTranslate.py
Infix expression to translate: A+B-C
The postfix representation of A+B-C is: A B + C -
$ python3 PostfixTranslate.py
Infix expression to translate: A+B*C
The postfix representation of A+B*C is: A B C * +
$ python3 PostfixTranslate.py
Infix expression to translate: A*(B+C)
The postfix representation of A*(B+C) is: A B C + *
$ python3 PostfixTranslate.py
Infix expression to translate: A | B*C^D % E - F/G + H & J
The postfix representation of A | B*C^D % E - F/G + H & J is: A B C D
^ *
E % F G / - H + J & |
```

The last example uses all the operators the program knows about and shows how it interprets their precedence and ignores whitespace. The `PostfixTranslate.py` program doesn't check the input for errors. If you type an incorrect infix expression, such as one with consecutive operators or unbalanced parentheses, the program will provide erroneous output.

Experiment with this program. Start with some simple infix expressions and see whether you can predict what the postfix will be. Then run the program to verify your answer. Pretty soon, "a postfix Jedi, you will become." Note that you can also use the InfixCalculator tool to practice these transformations, but you won't be able to use letters as variable names.

Evaluating Postfix Expressions

As you can see, converting infix expressions to postfix expressions is not trivial. Is all this trouble really necessary? Yes, the payoff comes when you evaluate a postfix expression. Before we show how simple the algorithm is, let's examine how a human might carry out such an evaluation.

How Humans Evaluate Postfix

Figure 4-12 shows how a human can evaluate a postfix expression using visual inspection along with a pencil and paper. The postfix expression, 345+×612+/−, is shown in the gray rectangle. In this simplified example, operands can be only single-digit integers.

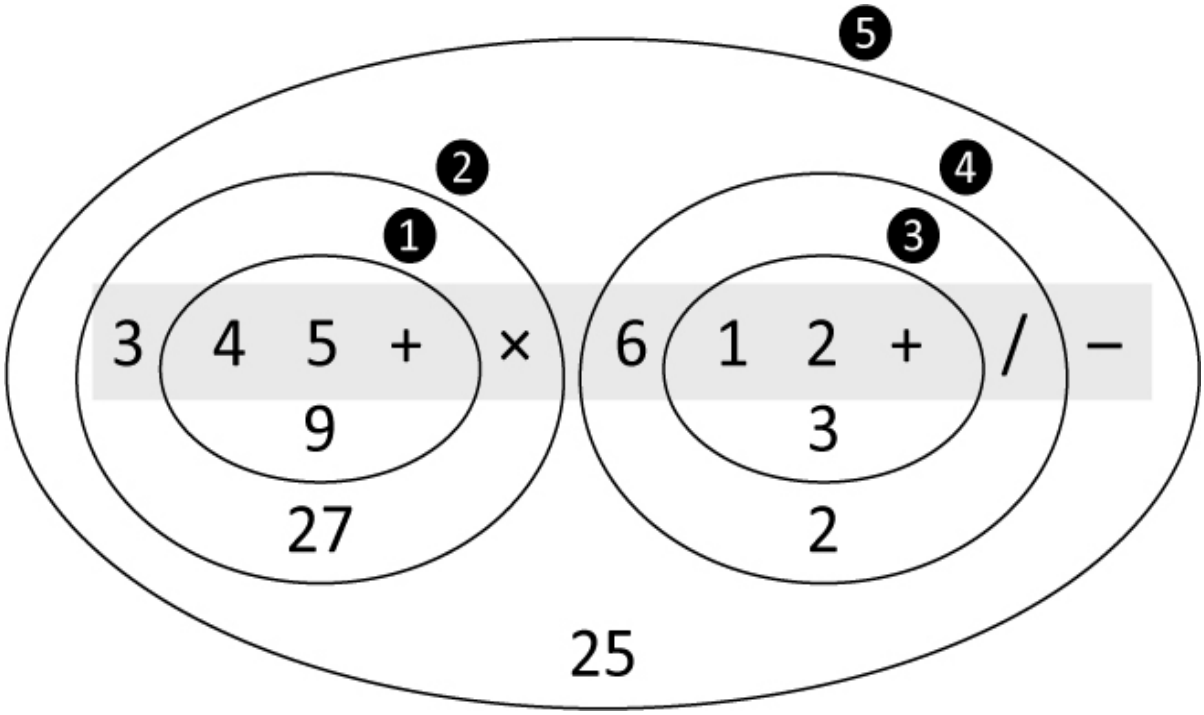

Figure 4-12 *Visual approach to postfix evaluation of 345+×612+/−*

Start with the first operator on the left and draw an oval around it plus the two operands to its immediate left. This is marked as step ❶ in the figure. Then apply the operator to these two operands—performing the actual arithmetic—and write down the result inside the oval. In the figure, evaluating 4+5 gives 9, which will be used step ❷.

Now go to the next operator to the right and draw an oval around it, the oval you already drew, and the operand to the left of that. Apply the operator to the previous oval and the new operand and write the result in the new oval, labeled ❷. Here 3×9 gives 27. Continue this process until all the operators have been applied: 1+2 evaluates to 3 in step ❸, and 6/3 is 2, in step ❹. The answer is the result in the largest oval at step ❺: 27−2 is 25.

Rules for Postfix Evaluation

How do you write a program to reproduce this evaluation process? As you can see, each time you come to an operator, you apply it to the last two operands you've seen. Remembering what the `PostfixTranslate.py` program does (Listing 4-10), suggests that it might be appropriate to store the operands on a stack. The approach for postfix evaluation, however, differs from the infix-to-

postfix translation algorithm, where *operators* were stored on the stack. You can use the rules shown in Table 4-14 to evaluate postfix expressions.

Table 4-14 *Evaluating a Postfix Expression*

Item Read from Postfix	Action Expression
Operand	Push it onto the stack.
Operator	Pop the top two operands from the stack and apply the operator to them. Push the result.

When you're done, pop the stack to obtain the answer. That's all there is to it. This process is the computer equivalent of the human oval-drawing approach of Figure 4-12.

Python Code to Evaluate Postfix Expressions

In the infix-to-postfix translation, symbols (A, B, and so on) were allowed to stand for numbers. This approach worked because you weren't performing arithmetic operations on the operands but merely rewriting them in a different format. Now say you want to evaluate a postfix expression, which means carrying out the arithmetic and obtaining an answer. The input must consist of actual numbers mixed with operators.

As shown in Listing 4-11, the `PostfixEvaluate.py` program imports the `PostfixTranslate` module and uses its functions to translate an infix expression to postfix before evaluating it. It makes use of the same `nextToken()` function from that module, but this time it's applied to the translated postfix string. The fence post loop goes through each of the tokens in the postfix string. For operators, the left and right operands are popped off the stack, the operation is performed, and the result is pushed back on the stack. For operands, the string version of the operand is converted to an integer and pushed on the stack.

Listing 4-11 *The* `PostfixEvaluate.py` *Program*

```
from PostfixTranslate import *
from SimpleStack import *
```

```
def PostfixEvaluate(formula):  # Translate infix to Postfix and
                               # evaluate the result

   postfix = PostfixTranslate(formula) # Postfix string
   s = Stack(100)                      # Operand stack

   token, postfix = nextToken(postfix)
   while token:
      prec = precedence(token)  # Is it an operator?

      if prec:                  # If input token is an operator
         right = s.pop()        # Get left and right operands
         left = s.pop()         # from stack
         if token == '|':       # Perform operation and push
            s.push(left | right)
         elif token == '&':
            s.push(left & right)
         elif token == '+':
            s.push(left + right)
         elif token == '-':
            s.push(left - right)
         elif token == '*':
            s.push(left * right)
         elif token == '/':
            s.push(left / right)
         elif token == '%':
            s.push(left % right)
         elif token == '^':
            s.push(left ^ right)

      else:                     # Else token is operand
         s.push(int(token))     # Convert to integer and push

      print('After processing', token, 'stack holds:', s)

      token, postfix = nextToken(postfix) # Fence post loop

   print('Final result =', s.pop()) # At end of input, print result

if __name__ == '__main__':
   infix_expr = input("Infix expression to evaluate: ")
   print("The postfix representation of", infix_expr, "is",
         PostfixTranslate(infix_expr))
   PostfixEvaluate(infix_expr)
```

The `PostfixEvaluate()` method includes a print statement that shows the stack contents after each token is processed, plus a final print statement showing the answer. The "main" part of the program (when `__name__ == '__main__'`) prompts the user for an infix expression, prints its postfix representation, and then evaluates it. Running the program produces a result like this:

```
$ python3 PostfixEvaluate.py
Infix expression to evaluate: 3*(4+5)-6/(1+2)
The postfix representation of 3*(4+5)-6/(1+2) is 3 4 5 + * 6 1 2 + / -
After processing 3 stack holds: [3]
After processing 4 stack holds: [3, 4]
After processing 5 stack holds: [3, 4, 5]
After processing + stack holds: [3, 9]
After processing * stack holds: [27]
After processing 6 stack holds: [27, 6]
After processing 1 stack holds: [27, 6, 1]
After processing 2 stack holds: [27, 6, 1, 2]
After processing + stack holds: [27, 6, 3]
After processing / stack holds: [27, 2.0]
After processing - stack holds: [25.0]
Final result = 25.0
```

Note this is the same calculation as shown in Figure 4-12 although the Python interpreter produced a floating-point number when it performed the division operation. The `PostfixEvaluate.py` program has not added any error checking on the input, so if you give it invalid expressions or expressions that contain names instead of numbers, the results will be wrong.

Experiment with the program and with the InfixCalculator tool. Try different expressions and check to see that they translate into postfix as you expect and verify the evaluation process. Use the animation controls to slow down or pause the calculator to see how the steps are carried out. Trying out different experiments can give you an understanding of the process faster than reading about it. One difference between the code and the InfixCalculator tool is that the tool creates floating-point numbers when division is used and pushes it on the stack. If that floating-point number creates an error, however, the tool will convert it to an integer and try again.

Summary

- Stacks, queues, and priority queues are data structures usually used to simplify common programming operations.
- In these data structures, only one data item can be accessed. They are designed to work for specific patterns in the order of access to the items.
- A stack allows access to the last item inserted: last-in, first-out (LIFO).
- The important stack operations are pushing (inserting) an item onto the top of the stack and popping (removing) the item that's on the top.
- A queue allows access to the first (oldest) item that was inserted: first-in, first-out (FIFO).
- The important queue operations are inserting an item at the rear of the queue and removing the item from the front of the queue.
- Stacks and queues typically do not have search or traverse operations because they are not databases.
- Stacks and queues typically do support a peek operation to examine the next item to be removed.
- A queue can be implemented using a circular array, which is based on a linear array in which the indices wrap around from the end of the array to the beginning.
- A deque is two-ended queue that allows insertion and removal operations at both ends.
- The frontmost item in a priority queue is always the highest in priority and is the oldest in the queue of that priority.
- The important priority queue operations are inserting an item in sorted order based on priority and removing the oldest item within the highest-priority items.
- These data structures can be implemented with arrays or with other mechanisms such as linked lists.
- Ordinary arithmetic expressions are written in infix notation, so-called because the operator is written between the two operands.

- In postfix notation, the operator follows the two operands.
- Arithmetic expressions can be evaluated by translating them to postfix notation and then evaluating the postfix expression.
- A stack is a useful tool both for translating an infix to a postfix expression and for evaluating a postfix expression.

Questions

These questions are intended as a self-test for readers. Answers may be found in Appendix C.

1. Suppose you push the numbers 10, 20, 30, and 40 onto the stack in that order. Then you pop three items. Which one is left on the stack?
2. Which of the following is true?
   a. The Pop operation on a stack is considerably simpler than the Remove operation on a queue.
   b. The contents of a queue can wrap around, while those of a stack cannot.
   c. The top of a stack corresponds to the front of a queue.
   d. In both the stack and the queue, items removed in sequence are taken from increasingly higher index cells in the array.
3. What do LIFO and FIFO mean?
4. True or False: A stack or a queue often serves as the underlying mechanism on which an array data type is based.
5. As other items are inserted and removed, does a particular item in a queue move along the array from lower to higher indices, or higher to lower?
6. Suppose you insert 15, 25, 35, and 45 into a queue. Then you remove three items. Which one is left?
7. True or False: Pushing and popping items on a stack and inserting and removing items in a queue all take O(N) time.
8. A queue might be used appropriately to hold

a. the items to be sorted in an insertion sort.

b. reports of a variety of imminent attacks on the star ship Enterprise.

c. keystrokes made by a computer user writing a letter.

d. symbols in an algebraic expression being evaluated.

9. Inserting an item into a priority queue takes what Big O time, on average?

10. The term *priority* in a priority queue means that

a. the highest-priority items are inserted first.

b. the programmer must prioritize access to the underlying array.

c. the underlying array is sorted by the priority of the items.

d. the lowest-priority items are deleted first.

11. True or False: At least one of the methods in the `PriorityQueue.py` program in Listing 4-7 uses a linear search.

12. One difference between a priority queue and an ordered array is that

a. the lowest-keyed item cannot be removed easily from the array as the lowest-priority item can from the priority queue.

b. the array must be ordered while the priority queue need not be.

c. the highest-priority item can be removed easily from the priority queue but the highest-keyed item in the array takes much more work to remove.

d. All the above.

13. Suppose a priority queue class is based on the `OrderedRecordArray` class from Chapter 2. This allows treating the priority as the key and provides a binary search capability. Would you need to modify the `OrderedRecordArray` class to maintain removals as an O(1) operation?

14. A priority queue might be used appropriately to hold

a. passengers to be picked up by a taxi from different parts of the city.

b. keystrokes made at a computer keyboard.

c. squares on a chessboard in a game program.

d. planets in a solar system simulation.

15. When parsing arithmetic expressions, it's convenient to use

    a. a stack for operators and a queue for operands and to evaluate postfix using another stack.

    b. a priority queue for the operators and operands and to evaluate postfix using a stack.

    c. a stack to transform infix expressions into prefix expressions that are then evaluated with a queue.

    d. a queue to transform the operators into postfix and then evaluate them using a priority queue.

Experiments

Carrying out these experiments will help to provide insights into the topics covered in the chapter. No programming is involved.

4-A Start with the initial configuration of the Queue Visualization tool. Alternately insert and remove items. Notice how the front and rear indices rotate around in the queue. Does the inserted item ever get stored in the same cell twice?

4-B Think about how you remember the events in your life. Are there times when they seem to be stored in your brain like a stack? Like a queue? Like a priority queue?

4-C Consider various processing activities and decide which of the data structures discussed in this chapter would be best to represent the process. Some activities are

- Handling order requests on an e-commerce website
- Dealing with interruptions to the task you're working on (like this homework)
- Folding and putting away clothes to be worn next week
- Folding rags to be used in cleaning
- Triaging patients that arrive at hospitals or clinics during a disaster with a limited number of treatment rooms and doctors
- Following the different paths in a maze at each decision point

4-D Using the InfixCalculator Visualization tool, try entering some valid numeric expressions and some with errors. Try putting in unbalanced parentheses and operators with missing operands. The tool doesn't try to find your errors, but if you were going to change it to report errors, how would you do it? At what point in the processing are the errors discoverable?

Programming Projects

Writing programs to solve the Programming Projects helps to solidify your understanding of the material and demonstrates how the chapter's concepts are applied. (As noted in the Introduction, qualified instructors may obtain completed solutions to the Programming Projects on the publisher's website.)

4.1 Revise the `Stack` class in `SimpleStack.py` shown in Listing 4-1 to throw exceptions if something is pushed on to a full stack, or popped off an empty stack. Write a test program that demonstrates that the revised class properly accepts items up to the original stack size and then throws an exception when another item is pushed.

4.2 Create a program that determines whether an input string is a palindrome or not, ignoring whitespace, digits, punctuation, and the case of letters. Palindromes are words or phrases that have the same letter sequence forward and backward. Show the output of your program on "A man, a plan, a canal, Panama." You should use a `Stack` as part of the implementation as was shown in the `ReverseWord.py` program in Listing 4-3.

4.3 Create a `Deque` class based on the discussion of deques (double-ended queues) in this chapter. It should include `insertLeft()`, `insertRight()`, `removeLeft()`, `removeRight()`, `peekLeft()`, `peekRight()`, `isEmpty()`, and `isFull()` methods. It needs to support wraparound at the end of the array, as queues do.

4.4 Write a program that implements a stack class that is based on the `Deque` class in Programming Project 4.3. This stack class should have the same methods and capabilities as the `Stack` class in the `SimpleStack.py` module (Listing 4-1).

4.5 The priority queue shown in Listing 4-7 features fast removal of the highest-priority item but slow insertion of new items. Write a program

with a revised `PriorityQueue` class that has fast, O(1), insertion time but slower removal of the highest-priority item. Write a test program that exercises all the methods of the `PriorityQueue` class. Include the method that shows the `PriorityQueue` contents as a string and use it to show the contents of some test examples that include cases where the insertions do not occur in priority order.

4.6 Queues are often used to simulate the flow of people, cars, airplanes, transactions, and so on. Write a program that models checkout lines/queues at a store, using the `Queue` class (Listing 4-5). The system should model four checkout lines that initially start empty, labeled A, B, C, and D. Use a string to model the arrival and checkout completion events by using a lowercase letter to indicate a new customer arriving in one of the lines, and an uppercase letter to indicate a customer completing checkout from the named line. For example, the string `"aababbAbA"` shows three people going into the A checkout line with two being processed, while four people enter the B checkout line and none are processed. When a customer is added, put a "person" in the queue by adding a string like `"C1"` to the queue where the number increases for each new person. Any nonalphabetic character in the string (such as a space or comma) signals that the current content of each of the queues should be printed. Use an `OrderedRecordArray` from Chapter 2 to store the `Queue`s and their labels. Print error messages for queues that overflow or underflow. Show the output of your program on these strings:

```
aaaa,AAbcd,
abababcabc,Adb,Adb,Ca,
dcbadcbaDCBA-dddAcccBbbbCaaaD-
```

4.7 Extend the capabilities of `PostfixTranslate.py` in Listing 4-9 and `PostfixEvaluate.py` in Listing 4-11 to include the infix assignment operator, $A = B$. When evaluating expressions on the stack that reference variables, look up the assigned variable values before performing numeric operations. The assignment operator has the lowest precedence of any of the other operators. Unlike Python, the assignment operator itself should return the right-hand side value as its result. In other words, `A=3*2` should return a value of 6 (in Python, it returns `None)` with the side effect of binding A to 6. In this extended evaluator, references to variables must occur after they have been set (in some

higher-precedence expression to the left of the reference). Your program should print an error message if the expression references variables that are not set. Variable values should be retrieved when an operator is trying to use the value for a calculation (not when they are pushed on the stack). Your program should display the contents of the stack as it processes each token. For example:

```
$ python3 project_4_7_solution.py
Infix expression to evaluate: (A = 3 * 2) * A
The postfix representation of (A = 3 * 2) * A is: A 3 2 * = A *
After processing A stack holds: [A]
After processing 3 stack holds: [A, 3]
After processing 2 stack holds: [A, 3, 2]
After processing * stack holds: [A, 6]
After processing = stack holds: [6]
After processing A stack holds: [6, A]
After processing * stack holds: [36]
Final result = 36
```

The first appearance of *A* defines the value for *A* as 6, and the second appearance references that value. Use the `OrderedRecordArray` class from Chapter 2 to store and retrieve records containing a variable name and value to implement this. Show the result of running your program on `'(A = 3 + 4 * 5) + (B = 7 * 6) + B/A'`.

5. Linked Lists

In This Chapter

- Links
- A Simple Linked List
- Double-Ended Lists
- Linked List Efficiency
- Abstract Data Types and Objects
- Ordered Lists
- Doubly Linked Lists
- Circular Lists
- Iterators

In Chapter 2, "Arrays," you saw that arrays had certain disadvantages as data storage structures. In an unordered array, searching is slow, whereas in an ordered array, insertion is slow. In both kinds of arrays, deletion is slow. Also, the size of an array can't be changed after it's created. If a larger or smaller array is needed, a new array can be created, but all the items it contains need to be copied into the new structure, which is slow.

In this chapter we look at a data storage structure that solves some of these problems: the **linked list**. Linked lists are probably the second most commonly used general-purpose storage structures after arrays.

The versatile linked list mechanism suits many kinds of general-purpose databases. It can also replace an array as the basis for other storage structures such as stacks and queues. In fact, you can use a linked list in many cases in

which you use an array, unless you need frequent random access to individual items using an index.

Linked lists aren't the solution to all data storage problems, but they are surprisingly useful and conceptually simpler than some other popular structures such as trees. We investigate their strengths and weaknesses as we go along.

In this chapter we look at simple linked lists, double-ended lists, ordered lists, doubly linked lists, and circular lists. We also examine the idea of abstract data types (ADTs), see how stacks and queues can be viewed as ADTs, discuss how they can be implemented as linked lists instead of arrays, and introduce iterators as data structures for traversing other data structures.

Links

In a linked list, each data item is embedded in a **link**. A link represents one element of the overall list. Each link holds some data and a way to get to the next link in the list. This can be done in a couple of ways. Figure 5-1 shows the two most common methods with a sample list of ingredients (used perhaps as a shopping list). In both cases, the data for each link is a record. The example has records with fields that could be named: *ingredient type*, *subtype*, *amount*, and *unit*. The difference between the two styles of lists is in how the path to the next link is stored. In both cases, the path is stored as a **reference**—a pointer leading to another record. That reference can either be a field in the data record itself or as field in a two-field structure designed as a general-purpose list.

Figure 5-1 *Different methods of representing links in a list*

In the first method where records are linked into a list as shown on the left of Figure 5-1, each record has a field at the right called something like *next*. The last record in the list doesn't have a *next* link, so that field must get some special value that cannot be confused with a reference to another record. The figure shows it as an empty box. In Python, you typically use `None` to represent no link. In Java, you use `null`. For all but the last record, the next field has a reference to another link record, represented by a curving arrow in the figure.

The second method uses a separate, two-field record to represent each link. The first field of each link record is a reference to the ingredient record, and the second field is a reference to the *next* link record. Those different kinds of references are shown as different colored arrows in Figure 5-1. One advantage of this method is that the records used in the list don't need to have their own *next* link; that information is stored in the two-field link record. There are disadvantages too: you must follow a reference from the link record to get to the ingredient (data) record, and this method uses a little more memory to store the full list. Those disadvantages usually are not significant. Another advantage of the linked list of records shown on the right is that a single record could be part of more than one list; in the records linked into a list method (at the left of the figure), they can be part of only one list at any point in time.

With either method of representing the links, it's a good idea to have a separate object class for the linked list itself. Doing so allows the creation of empty lists —a data structure that can be modified to expand and contract to become empty—and enables utility functions like `len()` and `str()` in Python to be applied to object instances. We choose to use a `LinkedList` class to represent the overall list. The only thing to be stored in the `LinkedList` object is the reference to the first `Link` object. We could also store other attributes of the overall list, like its length, and we examine that topic later.

Let's look at the Python definitions of the classes `Link` and `LinkedList` that implement a linked list of records. Each `Link` has two fields: one for the data and one for the next link. The `LinkedList` needs only one attribute, and we name it `__first` because it points to the first `Link` object of the list, if there is one. We use the double underscore prefix, as usual, to indicate these are private fields. Listing 5-1 shows the constructor part of the definitions along with basic tests to see whether they are empty or the last link in the list.

Listing 5-1 *Constructors and Tests for Linked List Classes*

```
class Link(object):              # One datum in a linked list
   def __init__(self, data, next=None): # Constructor
      self.__data = data         # The datum for this link
      self.__next = next         # Reference to next Link

   def isLast():                 # Test if link is last in the chain
      return self.__next is None  # True if & only if no next Link

class LinkedList(object):        # A linked list of data elements
   def __init__(self):           # Constructor
      self.__first = None        # Reference to first Link

   def isEmpty():                # Test for empty list
      return self.__first is None # True if & only if no 1st Link
```

The tests for `isLast()` and `isEmpty()` are nearly identical. The difference is their meaning in the context of the thing they represent—a link in the chain or the overall chain, respectively.

Because they are defined in Python, you don't have to specify the data types of the attributes in the classes, but they are important. In statically typed languages like Java, each attribute would need a declaration of its type. That's straightforward for the `__first` attribute of the `LinkedList` class because it is a `Link` object or `None`. Inside the `Link` class, the type to use for the `__next` attribute is more complicated. Here you need to refer to the very class that you're in the process of defining. This kind of class definition is sometimes called **self-referential** because it contains an attribute of the same type as itself.

More precisely, you don't want to store another entire `Link` object in the `__next` attribute of the `Link` class. If you did, then the self-referential definition would really become an endless nesting doll. We revisit that concept in the next chapter, "Recursion," but before we get there, we need to examine how references are stored and managed.

One of the reasons for specifying data types is so that the overall memory size of an object can be computed. The type information tells the compiler how many bytes to allocate when creating a new instance of the object. Self-referential class definitions would seem to need an infinite amount of memory if they were to contain themselves. Instead, they store a **reference** to the object. A reference is typically implemented by *storing the address of the object in*

memory. That way, the amount of memory needed for the attribute is a fixed size because there is a maximum memory address.

The reference acts like the arrows in Figure 5-1, a path to find the referenced object. In some languages, these are called **pointers**, and following a reference or pointer is called **dereferencing**. There are some distinctions between memory addresses, pointers, and references, but we don't discuss those subtleties here. The important concept is that the `Link` records are connected into a chain by references that must be followed to traverse the whole structure. Compare that to the arrays where all the elements of the array were placed next to each other in memory so that you could find any of them by using an integer index.

References and Basic Types

You can easily get confused about references in the context of linked lists. The confusion can be compounded by dynamic typing in Python, which means you don't know whether a variable holds a value or a reference to a value. Let's review how references work.

Every programming language has a set of **basic data types**, sometimes called **primitive** data types or *primitives*. These data types all have known, fixed sizes and usually take up one or two "machine words" at most. Integers and Boolean values are basic data types in almost every programming language. The Booleans, of course, have only two possible values, and the integer's range depends on the number of bits in a machine word (and some languages provide ways of specifying smaller or larger ranges than the standard machine word). Floating-point numbers are also normally stored with a fixed number of bits, based on the machine word size.

More complex data types like arrays, strings, records, and user-defined classes usually take more than one or two machine words of storage. For these structures, the programming language typically allocates memory for the size of the structure in a section of memory set aside for such items. When the structures are passed between functions, instead of copying the entire structure, only references to the structure are passed. Even when the object will not be passed to other functions, the programming language may still implement the object with a reference and a separate data storage location. Data types that are passed between functions as references are often called **reference data types**. They are distinct from the basic data types.

In Python, it's not obvious which data is stored as a primitive type and which is stored as a reference type. In other languages, programmers must be explicit about declaring whether a variable should be stored as primitive or as a reference to some other data type. That's one of the reasons why Python is popular as both a first programming language and an easy-to-use language; programmers don't need to manage references explicitly. When you write code that implements data structures, referencing and dereferencing become more of an issue because you are concerned with how long it will take to perform operations, how much space will be needed, and sometimes where the data will be stored.

One quick test to see whether data is being passed as a reference instead of by copying its value is to pass it to a function that modifies the value. For example, consider this small Python program:

```
def increment(a, b):
    a += 1
    b[0] += 1

x = 1
y = [1]
increment(x, y)
print(x, y)
```

What will the program print? The value of the variable `x` is initialized to 1, and the value of the variable `y` is a one-element list/array containing 1. When `increment()` is called, it increments the value of `a` and the first element of `b`. Will that change the values of the variables `x` and `y`? Do you think it will print `1 [1]`? Or maybe `2 [2]`? The answer is not obvious.

In the case of `x`, the answer is no; its value is not changed. The `print()` function will print the value 1 for `x`, because when `increment()` was running, the `a` variable stored *a copy of* 1, not the exact same 1 that was stored in `x`. When 1 was added to the value of `a`, the resulting 2 was stored back in `a` and did not change the copy stored in `x`.

In the case of `y`, however, the answer is different. The value of `y` is a Python list. After `increment()` runs, the value printed for `y` is `[2]`. Python passes lists as references. Any changes to the list by the `increment()` function affect the value in the caller's environment because both the function and the calling environment *share* the object being referenced.